example, similar to what we did with the default configuration properties. One of the benefits is that it makes it simple to run each application on a local Kubernetes cluster during development, either directly or using Tilt. Depending on your requirements, you might decide to use one approach or the other. Both are valid and used in real-world scenarios.

## Patches vs. templates

Kustomize's approach to customizing configuration is based on applying patches. It's quite the opposite of how Helm works (https://helm.sh). Helm requires you to template every part of a manifest that you would like to change (resulting in non-valid YAML). After that, you can provide different values for those templates in each environment. If a field is not templated, you can't customize its value. For that reason, it's not rare to use Helm and Kustomize in sequence, overcoming each other's shortcomings. Both approaches have pros and cons.

In this book I decided to use Kustomize because it's natively available in the Kubernetes CLI, it works with valid YAML files, and it's purely declarative. Helm is more powerful and can also handle complex application rollouts and upgrades that Kubernetes doesn't support natively. On the other hand, it has a steep learning curve, its templating solution has a few drawbacks, and it's not declarative.

Another option is ytt from the Carvel suite (https://carvel.dev/ytt). It provides a superior experience, with support for both patches and templates, it works with valid YAML files, and its templating strategy is more robust. It takes a bit more effort to get familiar with ytt than Kustomize, but it's worth the effort. Because it treats YAML as a first-class citizen, ytt can be used to configure and customize any YAML file, even outside Kubernetes. Do you use GitHub Actions workflows? Ansible playbooks? Jenkins pipelines? You can use ytt in all those scenarios.

Let's consider Catalog Service. We already have the base deployment configuration composed with Kustomize. It's located within the project repository in a dedicated folder (catalog-service/k8s). Now let's define an overlay to customize the deployment for staging.